

Experiment # 01

Operations on Linear Arrays

Objectives:-

The objectives of this session are to understand the various operations on arrays in C++.

Software Tools:-

Dev C++ / Visual Studio

Theory:-

We have already studied array in our computer programming course. We would be using the knowledge we learned there to implement different operation on arrays.

Traversing Linear Arrays:-

Let A be the collection of data elements stored in the memory of the computer. Suppose we want to print the contents of each element of A or suppose we want to count the number of elements of A with a given property. This can be accomplished by traversing A that is by accessing and Processing each element of A exactly once.

The following algorithm traverses a linear array. The simplicity of the algorithm comes from the fact that LA is a linear structure. Other linear structures such as linked list can also be easily traversed. On the other hand the traversal of non-linear structures such as trees and graphs is considerably more complicated.

Algorithm:-

(Traversing a Linear Array) Here LA is a linear Array with lowerBound LB and upper Bound UB. This algorithm traverses LA.

Applying an operation PROCESS to each element of LA.

1. **[Initialize counter] Set $K=LB$.**
2. **Repeat Step 3 and 4 while $K \leq UB$.**
3. **[Visit element] Apply PROCESS to $LA[K]$.**
4. **[Increase Counter] Set $K=K+1$.**
End of Step 2 Loop
5. **Exit**

Inserting and Deleting:-

Let A be a collection of data elements in the memory of computer. “Inserting” refers to the operation of adding another element to the collection A and “deleting” refers to the operation of removing one of the elements from A. Here we discuss the inserting and deleting when A is a linear array.

Inserting an element at the “end” of the linear array can be easily done provided the memory space allocated for the array is large enough to accommodate the additional element. On the other hand suppose we need to insert an element in the middle of the array. Then on average half of the elements must be moved downward to the new location to accommodate the new element and keep the order of other elements.

Similarly deleting the element at the “end” of an array presents no difficulties but deleting the element somewhere in the middle of the array would require that each subsequent element be moved one location upward in order to fill up the array.

Algorithm:-

(Inserting into Linear Array) INSERT (LA, N, loc, ITEM)

Here LA is a linear array with c elements and loc is a positive integer such that $loc < N$. This algorithm inserts an element ITEM into the loc in LA. c is count of total elements available in the array.

1. **[Initialize Counter] Set $J=N$.**
2. **Repeat Step 3 and 4 while $J \geq loc$.**
3. **[Move Jth element downward] Set $LA [J+1] = LA[J]$.**
4. **[Decrease Counter] Set $J=J-1$.**
- End of Step 2 Loop.**
5. **[Insert element] Set $LA[loc]=ITEM$.**
6. **[Reset c] Set $c=c+1$.**
7. **Exit.**

Algorithm:-

(Deletion from a Linear Array) DELETE(LA, N, K)

Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm deletes an element ITEM from the Kth position in LA.

1. **[Initialize Counter] Set $J=K$.**
2. **Repeat Step 3 and 4 while $J < N-1$.**
3. **[Move Jth element upward] Set $LA [J] = LA[J+1]$.**
4. **[Increase Counter] Set $J=J+1$.**
5. **End of Step 2 Loop.**

6. [Reset c] Set $c=c-1$.

7. Exit.

Lab Tasks:-

1. Write a C++ program to implement all the above described algorithms and display the output for each.

Conclusion

Lab Instructor. Kaynat Rana